

UNIVERSITÀ DEGLI STUDI DI MODENA E
REGGIO EMILIA

DIPARTIMENTO DI INGEGNERIA "ENZO FERRARI"

Laurea Magistrale in Ingegneria Informatica

Cloud and Cybersecurity

Documentazione Debug Home Security System

Sviluppatore:

Mattia Compagnone

Anno Accademico 2025/2026

Introduzione

Il presente documento analizza i problemi tecnici critici emersi durante lo sviluppo di un sistema di sicurezza domestica distribuito basato su architettura embedded-host. Il progetto implementa un sistema real-time multi-task utilizzando FreeRTOS su microcontrollore Raspberry Pi Pico (RP2040), integrato con un sistema host Windows per analytics avanzate e monitoraggio remoto. L'architettura del sistema si divide in:

- **Componente Embedded:**

- **Hardware:** Raspberry Pi Pico (RP2040) con 264KB RAM;
- **Sensori:** PIR HC-SR501 (rilevamento movimento) e SW-420 (vibrazione);
- **Attuatori:** 3 LED per feedback visivo (rosso/verde/giallo);
- **RTOS:** FreeRTOS con scheduler preemptive e task multipli;
- **Comunicazione:** USB Serial verso sistema host;

- **Componente Host:**

- **Piattaforma:** Windows con threading nativo;
- **Database:** SQLite3 per persistenza eventi e analytics;
- **Interfaccia:** Web server HTTP con dashboard real-time;
- **Analytics:** Pattern recognition e threat level dinamico;

Durante l'implementazione sono emersi quattro problemi critici che hanno richiesto analisi approfondita e soluzioni ingegneristiche specifiche:

1. **Gestione della memoria:** allocazione heap eccessiva causava fallimenti sistematici;
2. **Problemi di compatibilità:** versioni FreeRTOS incompatibili con port RP2040;
3. **Problemi di Stack Overflow:** stack overflow e competizione per memoria limitata;
4. **Stabilità della comunicazione:** conflitti tra USB stack e task scheduler;

L'approccio adottato per il debug ha seguito una metodologia sistematica: implementazione hook FreeRTOS per monitoraggio real-time, test progressivo del firmware, monitoraggio delle risorse in uso ed identificazione delle cause primarie e dei problemi secondari.

La documentazione seguente presenta l'analisi dettagliata di ogni problema, le soluzioni implementate e i risultati ottenuti.

Gestione Memoria in FreeRTOS su RP2040

Il sistema non riusciva ad allocare sufficiente memoria per tutti i task presenti e veniva generato il seguente errore:

```
1 Malloc fallita - memoria insufficiente
```

Sono partito dalla seguente configurazione iniziale:

```
1 #define configTOTAL_HEAP_SIZE (80 * 1024)
```

con RAM totale del RP2040 pari a 264 KB. Inoltre erano presenti diverse allocazioni simultanee:

- **Heap FreeRTOS:** 80KB
- **Stack 6 task (4096 bytes ciascuno):** 24KB
- **TinyUSB buffers:** 20KB
- **Sistema SDK:** 30KB

La memoria totale utilizzata si aggirava quindi intorno ai 150-155 KB. Le conseguenze erano quindi il blocco del sistema durante l'inizializzazione dei task, il fallimento della comunicazione USB e l'attivazione costante dell'hook **vApplicationMallocFailedHook**. La soluzione che è stata implementata nel file *FreeRTOSConfig.h* è la seguente:

```
1 #define configTOTAL_HEAP_SIZE (28 * 1024) // 28KB invece  
    di 80KB  
2 #define configCHECK_FOR_STACK_OVERFLOW 2 // Abilita  
    monitoring  
3 #define configUSE_MALLOC_FAILED_HOOK 1 // Abilita  
    hook debug
```

Il risultato è stato il raggiungimento della stabilità del sistema per 6 task funzionanti, il ripristino della comunicazione USB e l'ottimizzazione dello spazio di memoria

Gestione Incompatibilità FreeRTOS con RP2040 Port

Il problema si verificava durante la fase di compilazione ed era il seguente:

```
1 error: 'xEventGroupSetBitsFromISR' was not declared in  
2 this scope
```

Il conflitto si è generato poiché le funzioni *Event Groups* non sono disponibili nel port specifico.

La soluzione implementata ha previsto due step: nel primo step è stato fatto un downgrade ad una versione compatibile

```
1 cd C:\pico-sdk\lib\FreeRTOS-Kernel  
2 git checkout V10.4.6
```

Nel secondo step è stato configurato in maniera corretta il file *FreeRTOSConfig.h*:

```
1 #define configUSE_EVENT_GROUPS 1  
2 #define INCLUDE_xEventGroupSetBitFromISR 1
```

A questo punto non ci sono più stati errori di compilazione relativi all'uso di queste funzioni.

Gestione Stack Overflow

Il sistema funzionava con firmware semplice ma si bloccava immediatamente col firmware FreeRTOS. Era presente anche un errore di allocazione della memoria a run time. Il problema derivava dalla dichiarazione dei task:

```
1 // CONFIGURAZIONE PROBLEMATICAMENTE
2 xTaskCreate(sensor_task, "SENSOR", 4096, NULL, 3, NULL);
   // 4KB
3 xTaskCreate(pattern_task, "PATTERN", 4096, NULL, 4, NULL)
   ; // 4KB
4 xTaskCreate(control_task, "CONTROL", 4096, NULL, 2, NULL)
   ; // 4KB
5 // erano presenti altri 3 task = 22KB solo di stack
```

Analizzando l'overhead si nota che:

- **6 task x 4KB:** 24KB solo per i task;
- **3 queue x 1KB:** 3KB;
- **3KB per task overhead;**

Il risultato è quindi di circa 30KB quando invece la memoria heap disponibile è solo di 28KB.

La soluzione adottata è la seguente:

```
1 xTaskCreate(sensor_task, "SENSOR", 2048, NULL, 3, NULL);
   // 2KB
2 xTaskCreate(control_task, "CONTROL", 2048, NULL, 2, NULL)
   ; // 2KB
3 xTaskCreate(led_feedback_task, "LED", 1024, NULL, 1, NULL)
   ); // 1KB
```

In totale sono stati impiegati 5KB stack + overhead, quindi circa 8KB utilizzati. Il design pattern adottato è quindi cambiato per permettere il corretto funzionamento del programma:

- **Sensor Task:** Acquisizione PIR/vibrazione;
- **Control Task:** State machine;
- **LED Task:** Feedback visuale real-time;

È stato semplificato il pattern analysis integrandolo nel control task per ridurre l'overhead di memoria.

Gestione Comunicazione USB

La comunicazione USB con FreeRTOS risultava instabile; inoltre il sistema embedded era funzionante ma l'host non riceveva dati ed erano presenti dei conflitti di timing tra scheduler e USB stack.

I problemi sono stati causati da:

- **TinyUSB buffering:** competizione per memoria con heap FreeRTOS;
- **Interrupt Priorities:** conflitti tra SysTick (FreeRTOS) e USB interrupts;
- **Printf overhead:** buffer USB saturati da output verboso;

Le soluzioni implementate sono state tre:

- **Ottimizzazione Output:**

```

1 // PRIMA - Output eccessivo
2 printf("LED_VERDE_attivo\n"); // Ogni 50ms =>
   spam
3
4 // DOPO - Output solo ai cambi di stato
5 static security_state_t last_state =
   SECURITY_DISARMED;
6 if (current_state != last_state) {
7     printf("LED%s_attivo\n", (current_state ==
   SECURITY_ALARM) ? "ROSSO" : "VERDE");
8     last_state = current_state;
9 }

```

- **Stabilizzazione Hardware:**

```

1 // Periodo stabilizzazione PIR realistico
2 printf("[SENSOR] Stabilizzazione_PIR_in_corso...\n");
3 vTaskDelay(pdMS_TO_TICKS(5000)); // 5 secondi
   invece di loop immediato

```

- **Sistema Host Robusto:**

```

1 // Fallback automatico comunicazione
2 if (!serial.auto_detect_pico_port()) {
3     std::cout << "Modalita' simulata automatica"
   << std::endl;
4     // Sistema continua con dati simulati
5 }

```

Questo approccio ha portato al corretto funzionamento del sistema senza generare più problemi relativi al collegamento USB.

Conclusioni

Il progetto ha richiesto la risoluzione di quattro bug critici tipici dello sviluppo embedded su sistemi resource-constrained. L'analisi sistematica e le soluzioni implementate hanno trasformato un sistema instabile in una piattaforma robusta e performante.

Risultati di Performance

L'ottimizzazione della gestione memoria ha prodotto miglioramenti significativi nell'utilizzo delle risorse del sistema. L'heap FreeRTOS è stato ridotto da 80KB (30.3% della RAM totale) a 28KB (10.6%), mentre l'allocazione degli stack dei task è passata da 24KB a soli 5KB, rappresentando una riduzione del 79% dell'overhead. Questo ha permesso di aumentare la memoria disponibile per il sistema da 110KB a 200KB, corrispondente a un incremento dell'82%. La stabilità del sistema ha raggiunto livelli eccellenti, con un uptime continuo testato superiore ai 30 minuti senza errori o crash. Il task scheduling opera correttamente con 3 task concorrenti che mantengono le frequenze programmate. Le performance real-time del sistema soddisfano pienamente i requisiti di responsività. La latenza dall'evento fisico (movimento PIR o vibrazione) al feedback visivo LED risulta molto bassa, garantendo una reazione immediata del sistema. Il tempo di stabilizzazione del sensore PIR è stato impostato a 5 secondi, un valore realistico per sensori hardware commerciali. Il throughput della comunicazione USB mantiene stabilità costante senza disconnessioni, mentre il response time del sistema host per l'auto-rilevamento del dispositivo è inferiore ai 2 secondi.

Architettura Finale

Il sistema embedded finale opera con 3 task FreeRTOS caratterizzati da priorità bilanciate che garantiscono scheduling deterministico. La gestione robusta degli errori è assicurata da hook dedicati che monitorano in tempo reale le condizioni critiche del sistema. La comunicazione USB beneficia di buffer ottimizzati che eliminano i conflitti precedentemente riscontrati con il task scheduler. Il sistema host implementa un meccanismo di fallback automatico che permette il funzionamento sia in modalità reale (con Pico connesso) che simulata, garantendo operatività continua indipendentemente dalla disponibilità dell'hardware embedded. Il database SQLite strutturato in 3 tabelle dedicate (eventi, allarmi, system_log) fornisce una base solida per analytics avanzate. Il web server HTTP operante sulla porta 8080 espone API REST per l'interfacciamento con la dashboard, mentre il sistema di threading parallelo gestisce simultaneamente comunicazione seriale, web server e analytics senza interferenze.

Impatto delle soluzioni

Le soluzioni implementate hanno risolto completamente i problemi iniziali, trasformando fallimenti di allocazione memoria in un sistema stabile con il 68.6% di RAM disponibile, errori di compilazione in un build process completamente funzionante, crash runtime in uptime prolungato senza interruzioni, e instabilità di comunicazione in trasferimento dati affidabile con fallback robusto. Il sistema implementato fornisce una base solida per estensioni future, inclusa l'integrazione microROS reale per comunicazione ROS2 nativa, l'espansione di sensori tramite bus I2C per rilevamento di temperatura, umidità e accelerazione, l'implementazione di OTA updates per il firmware embedded, e l'integrazione di machine learning edge per pattern recognition avanzato.